Assignment 1

EN4554 - Deep Learening for Vision

Overview

This module has two parts. **Part 1** is a small research exercise: you will re-implement a LoRA variant on a vision–language model and test whether a *conceptual claim* of the paper that introduced it still holds in the vision domain. **Part 2** is a fully-specified engineering task: you will LoRA-finetune a small language model to call tools and measure the improvement.

Compute constraint

Everything in this module is designed to run inside a **free Google Colab** session. See Section 3.

1 LoRA Finetuning for Vision Language Models

Work through the five steps below.

Step 1 : Choose a LoRA variant

Pick **one** paper from the list given below.

1. DoRA [4]
2. PiSSA [5]
3. MiLoRA [7]
4. VeRA [3]
5. LoRA+ [1]
6. AdaLoRA [8]
7. MoRA [2]
8. HydraLoRA [6]

OR

Choose a LoRA variant paper to your own liking, that has been implemented for natural language tasks (and not vision language tasks). To make things easier for you, choose a model with `peft` (huggingface) integration.

Step 2 : Choose a VLM

Size rule

The model you apply LoRA to must have $\leq$ 500M parameters.

Recommended (all load and finetune in `transformers`):

- **SmolVLM-256M** — Default choice.
- **SmolVLM-500M** — if you prefer a little more capacity; still trivially fits free Colab.
- **Florence-2-base** (∼230M)

Note: The models are expected to perform on Visual Question Answering (VQA) tasks. Thus, models trained on *contrastive* objectives such as CLIP are unsuitable.

Step 3 : Identify a vision language experiment

- Read the chosen paper and identify interesting conceptual ideas that are supported by numerical results.
- Choose one of these ideas and translate it into a vision language hypothesis.

Step 4 : Choose a dataset

Pick a small task that is suitable to test your hypothesis and use a **subset** that fits a Colab session (a few thousand examples): TextVQA or DocVQA (SmolVLM is strong on document/OCR-style tasks), OK-VQA, a VQAv2 subset, or COCO / Flickr8k captioning. State clearly which split and how many examples you use.

Step 5 : Test whether the claim transfers to vision

Perform relevant experiments to prove or disprove the hypothesis. Crucially, your evaluation metric must be tied to the claim, not just overall accuracy.
For bonus points: Show a genuinely interesting interpretation or architectural addition that is uniquely applicable to the vision domain.

Step 6 : Present your results in a report

In your report, write down,

1. The central claim of the paper you chose. (One or two sentences)
2. The specific concept that you chose to translate to the vision language domain and the results table/figure that supports the concept.
3. The translated hypothesis.
4. Experimental setup including the pretrained model, the dataset, and other important choices with justifications if necessary.
5. Experimental outcome and interpretation, including a baseline vs adapted comparison.

Part 1 deliverables

(a) A short report ($\leq$ 3 pages): as briefed in Step 6.

(b) Runnable code (notebook or repo) that reproduces your main result.

2 LLM Agents: Tool Calling

An agent is an LLM in a loop: it decides on a step, calls a tool, reads the result, and repeats until the goal is met (Goal → Plan → Act → Observe → Done). The loop only works if the model can emit a correct tool call and knows when not to call one. In this short exercise you will build a minimal agent, LoRA-finetune the model inside it to call tools, and see what changes. You choose the model, tools and data. Part 2 is deliberately small: aim for about one Colab session (roughly 1 hour of GPU time).

Step 1: Build a minimal agent

- Pick a small instruction-tuned LLM ($\leq$ 1B parameters) whose chat template supports tools, e.g. Qwen2.5-0.5B-Instruct, Qwen3-0.6B or SmolLM2-360M-Instruct.
- Define 3–5 simple Python tools of your choice (e.g. calculator, unit converter, a fake weather or price lookup, a calendar). Using made-up data for lookups ensures the model must call the tool rather than answer from memory.
- Write an agent loop: generate, parse any tool call, execute it, feed the result back, and stop when the model gives a plain-text answer (or after a small maximum number of steps).

Step 2: Data

Create a small dataset of user requests with the correct tool call(s) and final answers, either by generating it from templates or by taking a subset of a public function-calling dataset (e.g. glaiveai/glaive-function-calling-v2). Include some requests that need no tool. Keep a held out test set of about 100 requests, written with different phrasings from the training set. State the sizes you use.

Step 3: Baseline vs. LoRA

Evaluate the untouched model in your agent loop, then LoRA-finetune it (loss on the assistant turns only) and evaluate again on the same test set. Report at least:

- **call validity:** the output parses as a tool call in the expected format;
- **call accuracy:** right tool and right arguments;
- **abstention:** correctly answering without a tool when none is needed;
- **task success:** the agent's final answer is correct (end to end).

Step 4: Failure analysis

Look at the remaining failures of both models (at least 10 each). Group them into a few categories (e.g. wrong format, wrong tool, wrong argument, unnecessary call, ignored the tool result) and say which ones LoRA fixed, which it did not, and which it introduced. In two or three sentences, discuss whether a better prompt alone might have been enough.

Part 2 deliverables

(a) A short write-up ($\leq$ 1 page): your setup (model, tools, data sizes, LoRA settings), the baseline-vs-LoRA table, and the failure analysis.

(b) A notebook that runs the agent, the training and the evaluation.

3 Practical Notes

Everything here fits a free-tier T4 (16 GB VRAM). GPU memory is *not* the binding constraint at these model sizes; session time and data size are.

- **Precision:** use `fp16`, not `bf16` — the T4 does not support bf16 in hardware. If you see NaNs, lower the learning rate.
- **Session limits:** free sessions disconnect on idle (∼90 min) and cap around ∼12 h with fluctuating quotas. Keep runs short (small subset, few epochs) and **checkpoint to Google Drive** so a disconnect is not fatal.
- **Data size:** full image datasets will not fit local disk/RAM — always use a subset and state its size.
- **Reproducibility:** fix and report your random seed.

References

[1] S. Hayou, N. Ghosh, and B. Yu. Lora+: Efficient low rank adaptation of large models. *arXiv preprint arXiv:2402.12354*, 2024.

[2] T. Jiang, S. Huang, S. Luo, Z. Zhang, H. Huang, F. Wei, W. Deng, F. Sun, Q. Zhang, D. Wang, et al. Mora: High-rank updating for parameter-efficient fine-tuning. *arXiv preprint arXiv:2405.12130*, 2024.

[3] D. Kopiczko, T. Blankevoort, and Y. Asano. Vera: Vector-based random matrix adaptation. In *International Conference on Learning Representations*, volume 2024, pages 6815–6835, 2024.

[4] S.-Y. Liu, C.-Y. Wang, H. Yin, P. Molchanov, Y.-C. F. Wang, K.-T. Cheng, and M.-H. Chen. Dora: Weight-decomposed low-rank adaptation. *arXiv preprint arXiv:2402.09353*, 2024.

[5] F. Meng, Z. Wang, and M. Zhang. Pissa: Principal singular values and singular vectors adaptation of large language models. *Advances in Neural Information Processing Systems*, 37:121038–121072, 2024.

[6] C. Tian, Z. Shi, Z. Guo, L. Li, and C. Xu. Hydralora: An asymmetric lora architecture for efficient fine-tuning. *Advances in Neural Information Processing Systems*, 37:9565–9584, 2024.

[7] H. Wang, Y. Li, S. Wang, G. Chen, and Y. Chen. Milora: Harnessing minor singular components for parameter-efficient llm finetuning. In *Proceedings of the 2025 Conference of the Nations of the Americas Chapter of the Association for Computational Linguistics: Human Language Technologies (Volume 1: Long Papers)*, pages 4823–4836, 2025.

[8] Q. Zhang, M. Chen, A. Bukharin, N. Karampatziakis, P. He, Y. Cheng, W. Chen, and T. Zhao. Adalora: Adaptive budget allocation for parameter-efficient fine-tuning. *arXiv preprint arXiv:2303.10512*, 2023.